

PLANO DE TESTES ATUALIZADO – SERVEREST API

Responsável: Letícia Vidal | Versão: Final Challenge v8.0 (Revisão Final)

1. 🎯 Objetivo do Projeto

O objetivo deste projeto é consolidar a estratégia de garantia de qualidade para a API ServeRest, realizando a transição de uma cobertura básica para uma suíte de testes automatizados de alta maturidade.

Através da utilização do Robot Framework em conjunto com técnicas de Engenharia de Prompt (GenAI), este desafio final visa:

- **Expandir a Cobertura de Testes:** Implementar mais de 40 cenários de teste, garantindo o cumprimento dos requisitos mínimos de 4 testes positivos, 4 negativos e 2 de contrato para cada um dos endpoints críticos (/usuarios, /login, /produtos, /carrinhos).
- **Validar a Integridade de Contratos e Regras:** Assegurar que as respostas da API estejam em conformidade com o esquema JSON esperado e que as regras de negócio (como controle de estoque e unicidade de usuários) sejam respeitadas.
- **Otimizar o Ciclo de Desenvolvimento (Shift-Left):** Utilizar Inteligência Artificial Generativa para acelerar a descoberta de cenários de borda (boundary values) e casos de erro complexos, reduzindo o tempo de design de testes.
- **Garantir Rastreabilidade e Manutenibilidade:** Estruturar o projeto seguindo padrões de Clean Code, com keywords reutilizáveis e massa de dados dinâmica (FakerLibrary), vinculando cada script automatizado a um ID de caso de teste no plano de planejamento.
- **Evidenciar a Qualidade:** Fornecer artefatos técnicos transparentes (Matriz de Cobertura Antes vs. Depois e Relatórios de Execução) que comprovem a resiliência do software para a entrega final da Sprint 4.

2. 📊 Matriz de Cobertura (Antes vs Depois)

Endpoint	Status	Positivo	Negativo	Contrato	Regras
Geral	🔴 ANTES	1	1	0	1
/usuarios	🟢 DEPOIS	4	5	2	2
/login	🟢 DEPOIS	3	4	3	2
/produtos	🟢 DEPOIS	3	5	2	2
/carrinhos	🟢 DEPOIS	2	8	5	3

3. 📌 MATRIZ DE CASOS DE TESTE (ATUALIZADA)

👤 /usuarios

ID	Cenário	Input	Resultado	Tipo
CT-U-01	Criar usuário com sucesso	Válidos	201 + _id	Positivo
CT-U-02	Email duplicado	Mesmo email	400 + message	Negativo
CT-U-03	Nome ausente	Sem nome	400	Negativo
CT-U-04	Email inválido	email inválido	400	Negativo
CT-U-07	Administrador inválido	admin=string	400	Contrato
CT-U-10	Persistência do usuário	GET /usuarios	Dados consistentes	Regra
CT-U-11	Não retornar senha no GET	GET	sem password	Segurança

🔑 /login

ID	Cenário	Input	Resultado	Tipo
CT-L-01	Login com sucesso	válidas	200 + token	Positivo
CT-L-02	Senha inválida	senha errada	401	Negativo
CT-L-06	Token presente no sucesso	—	auth string	Contrato
CT-L-08	Usar token endpoint protegido	GET prot.	200	Integração

📦 /produtos

ID	Cenário	Input	Resultado	Tipo
CT-P-01	Criar produto como admin	válidos	201 + _id	Positivo
CT-P-02	Usuário comum tentar criar	válidos	403	Negativo
CT-P-05	Preço negativo	preco=-10	400	Negativo
CT-P-07	Validar contrato sucesso	—	_id presente	Contrato
CT-P-09	Preço zero	preco=0	201*	Regra

🛒 /carrinhos

ID	Cenário	Input	Resultado	Tipo
CT-C-01	Criar carrinho com sucesso	válido	201	Positivo
CT-C-08	Estoque insuficiente	qtd alta	400	Regra
CT-C-09	Produtos vazio	[]	400	Contrato
CT-C-13	Tipo inválido quantidade	"2" (str)	400	Contrato

4. 🤖 Evidência GenAI (Amazon Q)

Prompt de Contexto Utilizado (Conforme Print):

```
Estou automatizando testes de API com Robot Framework para o endpoint /produtos da ServeRest.  
  
Contexto técnico fornecido à IA:  
- Arquivo: produtos.robot (test cases)  
- Arquivo: data_factory.resource (geração de massa com Faker)  
  
Comando: "Analise meus arquivos atuais e gere cenários de testes negativos que ainda não foram cobertos, focando em tipos de dados inválidos e valores de borda (boundary values) para os campos 'preco' e 'quantidade'. Retorne em formato de tabela."
```

Resumo do Output Gerado:

- A Amazon Q processou a estrutura das Keywords e propôs a expansão da suíte com foco em robustez de contrato:
- **Validação de Tipos:** Identificação da necessidade de testar strings em campos numéricos (preco/quantidade).
 - **Limites Físicos (Boundary):** Inclusão de testes para valores zerados e negativos que impactam a regra de negócio do estoque.
 - **Refino de Erros:** Sugestão de validação de mensagens específicas do servidor para campos ausentes no payload.

Tabela de Exemplos (Refino GenAI):

ID	Scenario	Input	Expected Result	Test Type
TC-P-N06	preco as string	{ preco: "dez" }	400 + response contains 'preco'	Contract/Type
TC-P-N08	preco as zero	{ preco: 0 }	400 (Check against contract)	Boundary/Neg
TC-P-N10	qtd as negative	{ quantidade: -1 }	400 (Must not be accepted)	Boundary/Neg

5. Check dos Requisitos e Boas Práticas

- ✓ **+10 novos testes:** ATENDIDO (44 totais) | ✓ 4 positivos, 4 negativos, 2 contrato por rota.
- ✓ **Boas práticas:** Keywords reutilizáveis, FakerLibrary (massa dinâmica), Asserts claros.
- ✓ **Organização:** Camadas (Resources, Data Factory, Base API).
- ✓ **Git:** Branch feature/challenge04-final-sprint4 | Commits semânticos.